CA2 精简版报告（给我们自己看）：异常感知的路由与封禁控制器

> 读完约 20 分钟，你应该能回答三件事：题目是什么、要我们做什么、我们是怎么做的、结果是多少。 所有数字来自我们自己的实验 R1（`results/validation-abc/`）。每个数字后面都有文件可以查。

0. 一分钟看懂

是什么？ 一个有 6 台交换机的网络。有台主机疯狂发包（洪泛攻击），把链路挤满，别人的正常流量就开始丢包。

要我们做什么？ 写一个控制器程序，让它自己做四件事：

1. 看：每 2 秒统计一次谁发包多、哪条链路忙；
2. 抓：发现某台主机发得远超它平时的水平，就认定它是攻击者；
3. 封：在所有交换机上加一条规则，把这台主机发出的包全部丢掉；
4. 绕：把被攻击的链路标成“很贵”，让别人的正常流量改走另一条路；攻击停了再走回来。

然后做实验、画图、解释图。

怎么打分（30 分）：

任务	分	一句话
1 拓扑	5	在 MiniEdit 里把题目给的拓扑搭出来并运行
2 监控、检测、封禁	5	算每个源的速率，用 EMA 基线检测，封掉攻击源，证明攻击包真的过不去
3 动态链路代价	10	每条链路的代价随“流量大小”和“有没有被攻击”变化
4 合法流量改路由	10	h1→h6 的流量在三种攻击下是否绕路，记录路径、丢包、吞吐，攻击结束后要恢复
5 性能评估	—	拓扑截图 + 三张图（代价、丢包、吞吐随时间变化）+ 文字解释

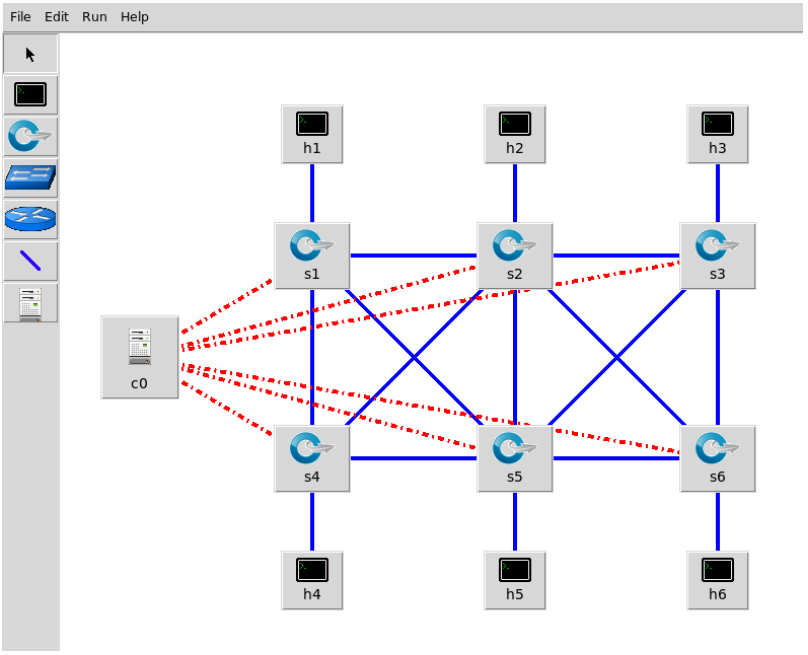
评分看三点：概念理解、设计推理（为什么这么设计）、结果解读（图说明了什么）。

我们交了什么： 控制器 `controller/integrated_controller.py`；一条命令跑完整个实验 `experiments/run_experiment.py`；报告 `report/report.pdf`；这次的结论一句话：所有丢包都发生在检测出来之前的 1.5–3.8 秒，封禁之后攻击包为 0；只有攻击打在 h1→h6 自己走的链路上时才改路由。

1. 名词小抄（先看这个）

名词	大白话
SDN	交换机自己不动脑，由一个“控制器”程序统一指挥怎么转发
控制器（Ryu）	我们写的程序，用 Python，跑在交换机外面
OpenFlow / 流表	控制器给交换机下命令的协议；流表 = 交换机里的“如果包长这样，就这样处理”的规则表
Mininet / MiniEdit	在一台电脑里模拟整个网络；MiniEdit 是它的画图界面
pps	每秒多少个包
EMA 基线	“它平时大概发多快”。每次新样本只占 30% 权重，慢慢跟着变
阈值	超过它就算异常。我们用 `max(3×基线, 基线+4×波动, 500 pps)`
hping3 –icmp –flood	攻击工具：不等回复，拼命发 ICMP 包
链路代价	给每条链路打的“难走程度”分数，越大越不想走
Dijkstra	在所有路里找“代价加起来最小”的那条的算法

2. 网络长什么样



MiniEdit 里搭的拓扑（`miniedit/my_topology.mn`，和题目的 `ceg_mesh_topo.py` 一致）

- s1–s6 是 6 台交换机，hi 接 si（IP `10.0.0.i`）。主机链路 40 Mbps、15 ms。
- 11 条交换机间的链路都是 40 Mbps，时延（ms）就是静态代价 C0：

链路	C0	链路	C0	链路	C0
s1-s2	5	s2-s3	10	s3-s6	7
s2-s4	6	s2-s5	12	s4-s5	15
s1-s4	8	s2-s6	14	s1-s5	18
s3-s5	9	s5-s6	20		

- 我们保护的合法流是 h1→h6。最短路 `s1-s2-s6`，代价 5+14 = 19；备用路 `s1-s2-s3-s6`，代价 5+10+7 = 22（只贵 3）。
- 题目要求依次攻击三条链路：

case	谁打谁	攻击实际压的链路	和 h1→h6 共用吗
a	h2→h3	s2-s3	否，不该改路由
b	h2→h6	s2-s6	是，应该改路由（主角）
c	h5→h6	s5-s3-s6（16 比直连 s5-s6 的 20 便宜，所以压的是 s3-s5 和 s3-s6）	否，不该改路由

3. 控制器每 2 秒做什么

测量 → 检测 → 定价 → 选路，循环不停。

1. 测量：问每台交换机要两样数据：每个源的包计数、每个端口的字节计数。
2. 检测：每个源有自己的 EMA 基线；速率超过阈值就判定为攻击者。
3. 封禁：在 6 台交换机上装丢包规则。
4. 定价：重新算每条链路的代价（Q6）。
5. 选路：对 h1→h6，比较“留在原路”和“最好的备选路”，备选便宜超过 2 才换。

4. 你问的 8 个问题

Q1. 怎么监控流量速率？

有两种速率，各用一种查询：

要什么	怎么查	怎么算
每个源（主机）的包速率，用来抓攻击者	OpenFlow FlowStats：每 2 秒问交换机每条流表项的包计数	速率 = (这次计数 − 上次计数) ÷ 间隔
每条链路的流量，用来算代价	OpenFlow PortStats：问每个端口的字节计数	速率 R = 两个方向较大的那个，字节差 × 8 ÷ 间隔，再做平滑

三个容易被问到的细节：

- 只在源主机连的第一台交换机上数。否则同一个包路上经过几台交换机就被数几次。
- 不数 ICMP echo reply。攻击 h6 时 h6 会回同样多的回复包，不排除的话，受害者 h6 会被误判成攻击者，被封掉后 h1→h6 也断了。
- 转发规则必须匹配“源 IP + 目的 IP + 协议”，FlowStats 里才有“源 IP”可以统计。

基线和检测：

- 每个源一个基线 b（EMA，权重 0.3）和波动 σ。阈值 T = max(3b, b+4σ, 500 pps)：`3b` 是题目给的规则；`b+4σ` 让阈值跟着这个源自己的波动走；500 是下限，防止 1 pps 的小流量被 3 倍规则误伤。
- 先比较、再更新基线：异常样本不学进基线。（老师给的参考代码是先更新再比较，实际阈值会变成约 21 倍。）
- 每个源最开始 5 个样本只学习、不报警（warm-up）。

代码：`controller/integrated_controller.py` 的 `_monitor`、`flow_stats_reply_handler`；`controller/sdn_logic.py` 的 `AnomalyDetector`。

Q2. 怎么配置的防火墙？Block Source IP？

不是 iptables，是 OpenFlow 流表规则。 检测到攻击者后，控制器给全部 6 台交换机各装一条：

> 匹配：IPv4 源地址 = 攻击者；动作：空（什么都不做 = 丢弃）；优先级 5000（比普通转发规则高，所以先命中）。

代码核心（`_block`）：

```
        match = parser.OFPMatch(eth_type=ether_types.ETH_TYPE_IP, ipv4_src=ip)
        self._add_flow(dp, PRIO_DROP, match, [], COOKIE_DROP)   # 动作为空 = 丢弃
```

- 在 Mininet 里查看规则：`mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s2 | grep 5000`，会看到：`priority=5000,ip,nw_src=10.0.0.2 actions=drop`，`n_packets` 一直在涨，说明攻击包在被丢。
- 控制器日志里的证据：`[EVENT] block h2->h3 {'ip': '10.0.0.2', 'pps': 5000.4, 'baseline': 94.26, 'threshold': 665.03, ...}`。
- 自动解封：攻击停止、drop 规则安静 10 秒后自动删规则。否则 case a 之后 h2 一直被封，case b 就没法复现。

Q3. 验证结果是什么？怎么验证“攻击流量不再被转发”？

用四种互相独立的证据，结论一致：

证据	在哪看	结果
① drop 规则的计数在涨	`dump-flows`	`n_packets` 从 18951 涨到 28521：攻击包在交换机上被丢
② 受害者上抓包	h3 上 `tcpdump`，只看来自 h2 的 ICMP	封禁后 3 秒内 0 个包
③ 攻击者自己的统计	`hping3` 结束时的汇总	发了 66,964 个，收到 0 个，100% 丢失
④ 整场实验的曲线	`fig5_blocking.png`	见下图

```
[EVENT] topology_ready -> {'detail_links': 11}
Host learned: 10.0.0.1 (00:00:00:00:00:01) at s1 port 1
Host learned: 10.0.0.2 (00:00:00:00:00:02) at s2 port 1
Host learned: 10.0.0.3 (00:00:00:00:00:03) at s3 port 1
Host learned: 10.0.0.4 (00:00:00:00:00:04) at s4 port 1
Host learned: 10.0.0.5 (00:00:00:00:00:05) at s5 port 1
Host learned: 10.0.0.6 (00:00:00:00:00:06) at s6 port 1
[EVENT] block h2->h3 {'ip': '10.0.0.2', 'pps': 5000.4, 'baseline': 94.26, 'threshold': 665.03
's2-s3']}
[EVENT] reroute h1->h3 {'reason': 'anomaly', 'old': 's1-s2-s3', 'new': 's1-s2-s5-s3', 'old_co
new_cost': 26.0}
```

```
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4 h5 h6
*** Adding switches:
s1 s2 s3 s4 s5 s6
*** Adding links:
(s6, h6)
*** Configuring hosts
h1 h2 h3 h4 h5 h6
*** Starting controlle
c0
*** Starting 6 switche

*** Starting CLI:

===== 1) connectivity
*** Ping: testing ping
h1 -> h2 h3 h4 h5 h6
h2 -> h1 h3 h4 h5 h6
h3 -> h1 h2 h4 h5 h6
h4 -> h1 h2 h3 h5 h6
h5 -> h1 h2 h3 h4 h6
h6 -> h1 h2 h3 h4 h5
*** Results: 0% droppe

===== 2) normal traff
PING 10.0.0.3 (10.0.0

--- 10.0.0.3 ping stat
15 packets transmitte
rtt min/avg/max/mdev =

===== 3) attack: h2 hp
[1] 10780

===== 4) drop rule ins
cookie=0xd0, duration=
cookie=0xd0, duration=

===== 5) attacker pack
0 packets received by
0 packets dropped by

===== 6) flood finishe
66964 packets transmit
round-trip min/avg/max
HPING 10.0.0.3 (h2-eth
```

手动演示截图：左边是控制器日志（block 事件），右边是 Mininet 控制台：drop 规则、tcpdump 0 包

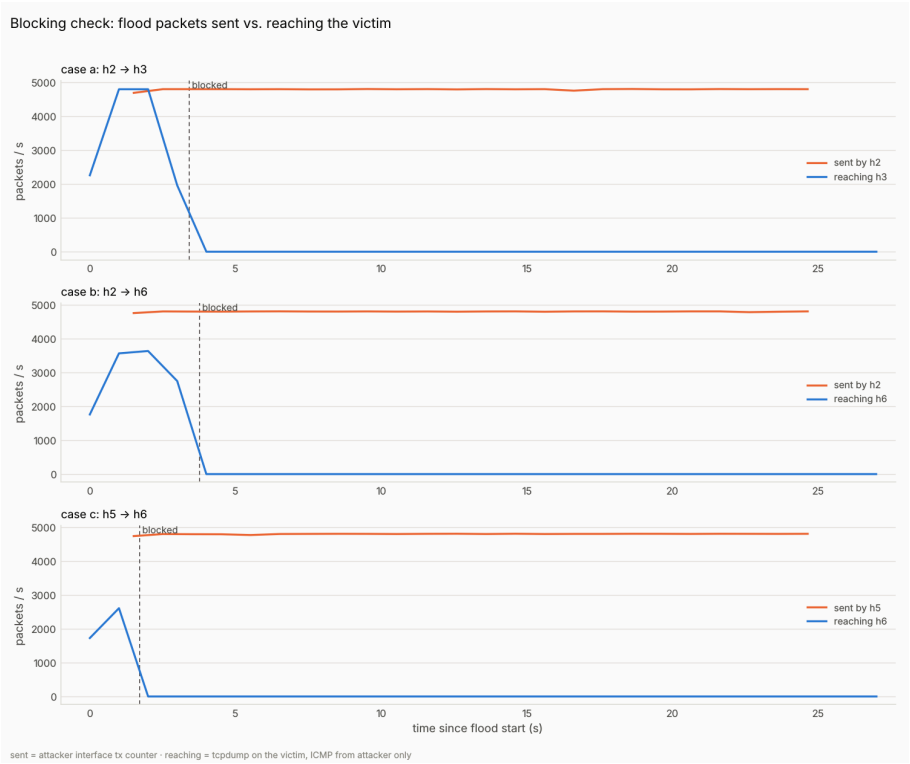


fig5：橙线 = 攻击者每秒发的包（一直约 4800），蓝线 = 到达受害者的包。虚线 = 封禁时刻，之后蓝线掉到 0 并保持

R1 的数字：

case	从洪泛开始到封禁（检测延迟）	封禁前到达受害者的攻击包	封禁后
a	3.4 s	13,779	0 pps
b	3.8 s	11,678	0 pps
c	1.7 s	4,242	0 pps

三次重复（9 次洪泛），检测延迟 1.5–3.8 秒，也就是 1–2 个采样周期。

Q4. 用 ping 计算基线的指令

为什么用 ping： 题目要求用 ping 模拟“正常流量”。ping 每秒 1 个包，所以每个源的基线 ≈ 1 pps（实测 0.9–1.1），阈值取下限 500 pps，正常小流量不会被误封。

手动做（在 Mininet CLI 里）：

```
mininet> pingall                                    # 先让控制器学会所有主机，应显示 0% dropped (30/30)
mininet> h2 ping -i 1 h3 > /tmp/h2ping.txt &        # 每秒 1 个包，建立 h2 的基线
mininet> sh sleep 15                                # 至少等 15 秒：前 5 个样本只学习
```

注意：

- 前 5 个样本（约 12 秒）是 warm-up，只学习不判断。所以基线阶段至少 15 秒。
- 一键脚本里的做法：h1→h6 跑 10 Mbps UDP，另外 5 对主机各 ping 1 pps：`h2→h4、h3→h5、h4→h1、h5→h2、h6→h3`，基线阶段 40 秒。
- 一个已知现象：R1 里 case b 时 h2 的基线是 148 pps、阈值 1046，因为 case a 洪泛刚开始的几个样本在封禁前被学进了基线；所以基线阶段要长，且“先比较再更新”能减少这种污染。
- 在有链路时延的 VM 里，pingall 变慢，warm-up 要加到 12 个样本，否则 h1 会被自己误封。脚本会自动设 `CEG_WARMUP=12`。

Q5. 模拟攻击流量的指令

```
mininet> h2 hping3 --icmp --flood -d 1000 h3 &      # case a：h2 攻击 h3（压 s2-s3）
mininet> sh sleep 6
mininet> h2 pkill hping3                            # 停止攻击
```

- `--icmp`：发 ICMP 包；`--flood`：不等回复，能多快发多快；`-d 1000`：每包带 1000 字节数据。
- 题目里写的 `-icmp` 是笔误，实际要 `--icmp`。
- 速度：约 4800 包/秒 ≈ 40 Mbps，刚好打满一条 40 Mbps 的链路。

另外两个 case 只换攻击者和目标：

case	命令
b	`h2 hping3 --icmp --flood -d 1000 h6 &`
c	`h5 hping3 --icmp --flood -d 1000 h6 &`

Q6. 每条链路的动态代价怎么算？结果是什么？

公式（题目的 Eq.1）：

```
C(t) = C0  +  α × R(t) / Rmax  +  β × A(t)
       静态       流量负载项          异常项
```

符号	含义	我们的取值
C0	静态代价 = 链路时延（ms）	见第 2 节的表
R(t)	这条链路现在的流量（PortStats 测量，做平滑）	实时
Rmax	链路最大速率	40 Mbps
α	负载项的权重	10（负载满时最多加 10）
A(t)	这条链路被攻击了吗（0 或 1）	见 Q7
β	异常项的权重	30

图里画的是“归一化代价” C / C0（1.0 = 没变化）。

以 s2-s6（C0 = 14）为例，一步步算：

阶段	R	A	C	C / C0
平时（只有 h1 的 10 Mbps）	10.8 Mbps	0	14 + 10×0.27 = 16.7	1.19
洪泛刚开始（攻击还没封）	29.6 Mbps	1	14 + 7.4 + 30 = 51.4	3.67（峰值）
封禁后，攻击者还在撞 drop 规则	约 0	1	14 + 0 + 30 = 44	3.14（平台）
攻击停止约 10 秒后	10.8 Mbps	0	16.7	1.18（恢复）

R1 里三种攻击对应链路的最终结果：

case	链路	C0	平时	峰值	平台（封禁后）	攻击结束后
a	s2-s3	10	1.0	4.61	4.0	1.0
b	s2-s6	14	1.19	3.67	3.14	1.18
c	s3-s5	9	1.0	4.54	4.33	1.0
c	s3-s6	7	1.0	5.51	5.29	1.0
c	s5-s6	20	1.0	1.0	1.0（攻击没走这条）	1.0

平台值正好是 (C0+30)/C0，例如 (14+30)/14 = 3.14。

Q7. 我们用什么方法提高链路成本？

三个手段，作用不同：

1. 负载项 α·R/Rmax：链路越忙越贵（最多 +10）。攻击一开始，负载项一下子把代价拉到峰值。
2. 异常项 β·A（+30）：这是主要手段。控制器封禁攻击者时，把“攻击者到受害者这条路径上的所有链路”标成 A = 1。
   - 封禁后攻击包被丢，链路负载很快降到 0，负载项会消失。如果只有负载项，被攻击的链路马上又变便宜，流量会走回去。 β 项就是“记忆”：只要攻击者还在撞 drop 规则，A 就保持为 1；停了以后再保持 10 秒，才恢复。
   - 所以封禁后代价停在平台值（3.14、4.0 等），而不是回到 1。
3. 选路时的两个保护：
   - what-if：评估备选路时，把这条流自己的速率加到它还没走的链路上，否则一换路就把新路压贵，来回振荡；
   - 备选路要便宜超过 2 才换（迟滞），避免频繁换路。

为什么 β = 30？ 备用路只比原路贵 3（22 比 19），β 要足够大，让被攻击链路明显比备用路贵。实验验证过：β = 0 或 2 时 case b 不会改路由，但封禁后丢包同样是 0。结论：止损靠封禁；β 的作用是让流量离开被攻击的链路，对封禁漏掉的情况（比如伪造源 IP）才是真正的后路。

Q8. Task 4：h1→h6 的 flow 怎么建？记录什么？数值多少？三条链路上发生了什么？

怎么建 h1→h6 的流：

```
h6:  iperf -s -u -i 1 -e                        # 接收端，每秒报告吞吐和丢包
h1:  iperf -c 10.0.0.6 -u -b 10M -t <秒> -i 1  # 发 10 Mbps 的 UDP，约 890 包/秒
h1:  ping -D -i 0.5 10.0.0.6                    # 每 0.5 秒 ping 一次，测往返时间 RTT
```

先 `pingall`，控制器学会主机位置后，用 Dijkstra 选最短路并把整条路径装进交换机。时间线：基线 40 秒 → 每个 case 洪泛 25 秒 → 恢复 45 秒。

记录什么，初始值是多少（R1，攻击前）：

记录项	来自哪个文件	初始数值
路径	`controller/paths.csv`（每 2 秒）	`s1-s2-s6`
路径代价	同上	静态 19；带上负载是 24.4
备用路径	—	`s1-s2-s3-s6`，22
各链路代价	`controller/links.csv`（每 2 秒）	s2-s3 = 1.0，s2-s6 = 1.19，s3-s6 = 1.0（归一化）
丢包	h6 的 `iperf_server.txt`（每秒）	0%
吞吐	同上	10.5 Mbps（892 包/秒）
RTT	`ping_h1_h6.txt`	约 0.6 ms（容器）；VM 带时延约 99.8 ms
改路由事件	`controller/events.csv`	—

三条被攻击的链路上发生了什么（R1）：

	a：s2-s3（h2→h3）	b：s2-s6（h2→h6）	c：s3-s5 / s3-s6（h5→h6）
检测并封禁	3.4 s，封 h2	3.8 s，封 h2	1.7 s，封 h5
被标记的链路	s2-s3	s2-s6	s3-s5、s3-s6
链路代价	1.0 → 4.61 → 平台 4.0	1.19 → 3.67 → 平台 3.14	4.54 / 5.51 → 平台 4.33 / 5.29
h1→h6 路径	不变 `s1-s2-s6`	改走 **`s1-s2-s3-s6`**，结束后切回	不变
丢包（最坏 1 秒）	0%	48%（检测前，逐秒 24/47/48/27%）	2.5%
吞吐（最低）	10.5 Mbps	5.4 Mbps（检测前），改路由后马上回 10.5	10.0 Mbps
连通性	一直通	一直通	一直通
结束后	代价回 1.0	洪泛结束约 12–15 秒后路径切回，代价回 1.18	代价回 1.0

为什么是这样（每列一句话）：

- a：h1→h6 根本不走 s2-s3，所以不受影响，也不该改路由。
- b：攻击打在 h1→h6 自己走的 s2-s6 上，40 Mbps 洪泛加 10 Mbps 合法流，链路被挤爆，丢包 48%。1.5–3.8 秒内被检测到、封禁的同一个周期里，控制器发现“留在原路代价 58.8，换路只要 30.0”，于是改走 s1-s2-s3-s6，丢包立即归零。洪泛结束约 12 秒后解封，A 清零，原路变成 24.5，备路 30.1，切回，一次洪泛正好改路由 2 次（去一次、回一次）。
- c：攻击者 h5 按代价走的是 s5-s3-s6，所以被标的是 s3-s5 和 s3-s6，不是 s5-s6（它的代价一直 1.0）。h1 的路径不经过这两条，不需要换。那 2.5% 的小丢包是因为所有攻击包都冲向 h6，h6 自己的接入链路是 h1 的流也必须经过的，绕不开，只能靠封禁收尾。

5. Task 5：图和解释

五张图都在 `results/validation-abc/figures/` 和 `report/figures/`：

图	对应题目	看什么
`fig1_link_cost.png`	5.3a 归一化链路代价 vs 时间	6 条链路的小图
`fig2_packet_loss.png`	5.3b 丢包 vs 时间	h1→h6 的逐秒丢包
`fig3_throughput.png`	5.3c 吞吐 vs 时间	h1→h6 的逐秒吞吐
`fig4_overview.png`	总览	路径、代价、丢包、吞吐、RTT 在同一条时间轴上
`fig5_blocking.png`	Task 2 证据	攻击者发送 vs 受害者收到

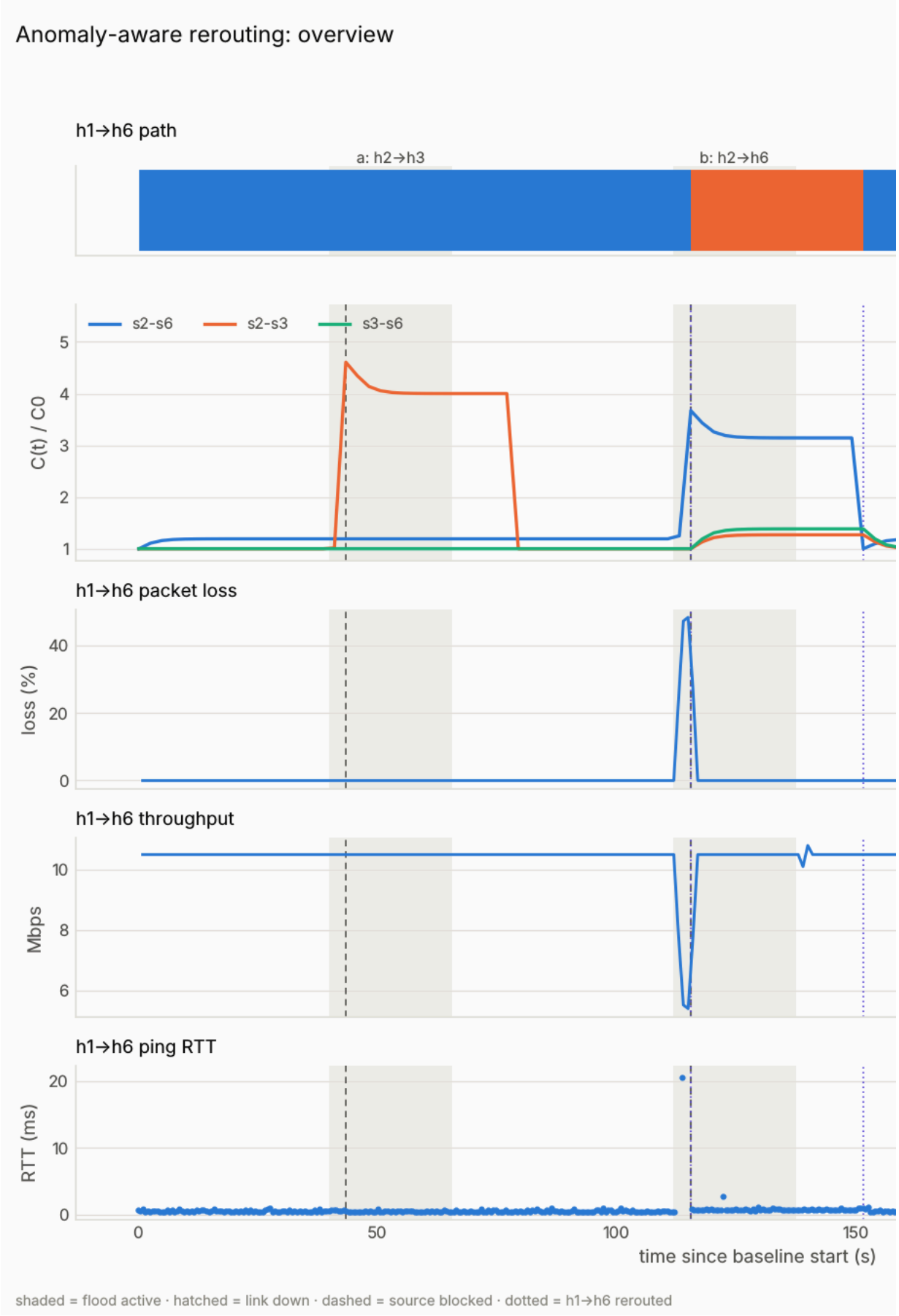


fig4 总览：阴影 = 洪泛期，虚线 = 封禁时刻，点线 = h1→h6 改路由

怎么读这张图（对应题目 5.4 要解释的四项）：

变化	解释
路由代价	洪泛开始，被攻击链路代价冲到峰值（负载项 + β 项）；封禁后负载项消失，只剩 β 项，停在平台；攻击停止约 10 秒后回到 1
路径选择	只有 case b（蓝条变橙条）改了路；a、c 的攻击链路不在 h1→h6 的路径上，蓝条不变
丢包	只在 case b 检测前那 2–4 秒出现尖峰（48%）；封禁和改路由发生的同一时刻丢包归零。a 为 0，c 只有约 2.5% 的小尖峰
吞吐	平时 10.5 Mbps；case b 检测前掉到 5.4 Mbps；改路由后立刻回到 10.5 Mbps

整场的主线（讲解时反复说这句）： 丢包全部发生在"被检测到之前"；封禁让攻击包归零；代价让合法流量离开被攻击的链路；迟滞让它只切回一次。

再多说一点（加分项）： 我们还在官方 Mininet VM 里带真实链路时延重跑过（`results/vm-delay-abc/`）：检测延迟 2.2 / 1.6 / 3.8 秒，行为和容器一致；h1→h6 的 RTT 基线 99.8 ms，绕路时 109 ms，切回后 100 ms。

6. 讲解时被追问怎么答

问题	一句话答案
为什么不直接用 3 倍规则？	固定 3 倍对小基线太敏感（1 pps 的 3 倍只有 3），对大流量又太宽松；我们加了波动项和 500 pps 的下限
为什么检测要 2–4 秒？	轮询间隔是 2.4 秒，第一个样本可能只覆盖一部分洪泛；这是最弱的一环
封了 h2 会不会误伤？	会，封禁期间 h2 的所有流量（包括它的 1 pps ping）都丢，所以 10 秒安静后自动解封
源 IP 是伪造的怎么办？	按源 IP 封禁就无效了；这时靠 β 项把流量绕开被攻击的链路，这是 β 的真正价值
攻击结束后怎么恢复？	drop 规则安静 10 秒 → 自动解封 → A 清零 → 代价回落 → 路径切回
为什么只改 1 次又切回，不会来回抖？	what-if 估算 + 便宜要超过 2 才换（迟滞）；每次洪泛正好 2 次改路由

更多问题和答案见 `docs/QA_Bank.pdf`（100 题）。

7. 文件和复现

要什么	在哪
控制器	`controller/integrated_controller.py`、`controller/sdn_logic.py`
一键实验	`sudo -E python3 experiments/run_experiment.py`（约 5 分钟，结果在 `results/<时间戳>/`）
现场演示	`bash demo/live_demo.sh`（四格画面，跑完出结果表）
本文所有数字的原始数据	`results/validation-abc/`（`summary.md`、`controller/*.csv`、`iperf_server.txt`、`flows/`）
手动演示（Task 2）的控制台输出	`results/manual-demo/mininet_console.txt`
最终报告	`report/report.pdf`
更详细的速读和设计决策	`docs/Project_Quick_Guide_zh.pdf`、`PLAN.md`